

PRESENTACIÓN INICIAL

Mi proyecto se llama RuedaFácil.

Resuelve problemas como, en algunas empresas de alquiler de vehículos no tienen un sistema informático para organizar clientes, coches y contratos. Es una aplicación de consola que resuelve este problema: permite gestionar todo el negocio sin necesidad de una interfaz gráfica cara, funcionando directamente desde el terminal.

Va dirigido a empresas de alquiler de vehículos que buscan una solución funcional, rápida y que almacene todos los datos en una base de datos MySQL.

OBJETIVOS DEL PROYECTO

Los objetivos principales del sistema, tal como aparecen en el repositorio de GitHub, son:

- **Gestión de clientes:** crear, modificar, eliminar clientes identificados por su DNI, y consultar su historial de alquileres.
- **Gestión de vehículos:** registrar nuevos vehículos, modificar su información, eliminarlos y consultar los disponibles por categoría.
- **Gestión de alquileres:** crear contratos, modificar datos, eliminar alquileres y gestionar los estados del contrato (Activo, Completado, Cancelado).
- **Gestión de categorías y empleados:** asociar vehículos con categorías y gestionar los empleados relacionados con los alquileres.
- **Gestión de excepciones:** el sistema incluye dos excepciones personalizadas: ClienteNoEncontradoException y VehiculoNoDisponibleException.

Todas estas funcionalidades están implementadas con un CRUD completo y se almacenan en una base de datos MySQL.

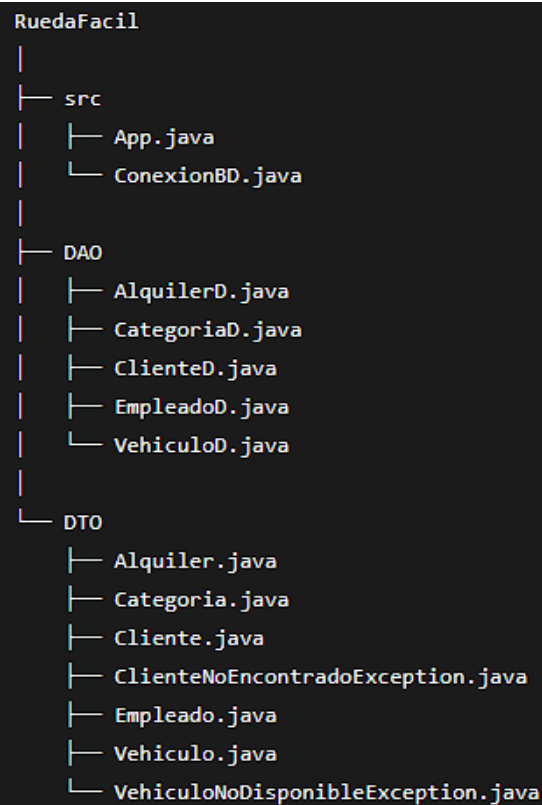
TECNOLOGÍAS UTILIZADAS

- Java JDK para la lógica de la aplicación.
- MySQL como sistema gestor de la base de datos relacional.
- JDBC (conector mysql-connector-j-8.4.0.jar) para la conexión entre Java y MySQL.
- Eclipse / VS Code como entornos de desarrollo.
- GitHub para el control de versiones y la publicación del código fuente.

Dentro de Java, he utilizado:

- Colecciones: HashMap, TreeSet, ArrayList, LinkedList.
- Streams con operaciones como filter(), map(), sorted() y collect().
- Comparable y Comparator para ordenar listas según diferentes criterios.
- Excepciones personalizadas para manejar errores de negocio.

DISEÑO DE LA APLICACIÓN



El proyecto sigue el patrón de diseño DAO y DTO:

- El DAO contiene la lógica de acceso a la base de datos. Por ejemplo, VehiculoD.java incluye los métodos crearVehiculo, modificarVehiculo, eliminarVehiculo y listarTodos, que utilizan sentencias SQL preparadas para interactuar con MySQL.
- El DTO define los objetos que transportan los datos. Por ejemplo, Vehiculo.java almacena matrícula, marca, modelo, año, combustible, plazas, precio por día, estado y categoría, con sus correspondientes getters y setters.
- ConexionBD.java gestiona la conexión con la base de datos con JDBC, con la URL, usuario y contraseña.

BASE DE DATOS

La base de datos se ha creado con MySQL. El script de creación completo lo muestro ahora.

Tablas:

CATEGORIA: id_categoria (PK), nombre_categoria

CLIENTE: dni (PK), nombre, telefono, email, direccion, num_carnet

EMPLEADO: id_empleado (PK), nombre, cargo, oficina, turno, anos_experiencia

VEHICULO: matricula (PK), marca, modelo, ano, combustible, plazas, precio_dia, estado, id_categoria (FK → CATEGORIA)

ALQUILER: id_alquiler (PK), fecha_inicio, fecha_devolucion_prevista, fecha_devolucion_real, id_cliente (FK → CLIENTE), id_empleado (FK → EMPLEADO), matricula (FK → VEHICULO), precio_total, estado_contrato

Relaciones:

Un cliente → muchos alquileres (1:N)

Un vehículo → muchos alquileres (1:N)

Un empleado → muchos alquileres (1:N)

Una categoría → muchos vehículos (1:N)

Elementos avanzados:

- **Procedimientos:** obtener_alquileres_por_estado (filtra alquileres por Activo, Completado o Cancelado) y calcular_alquileres_por_mes (genera un resumen mensual de alquileres para un año concreto).
- **Funciones:** mes_en_letras (convierte número de mes a nombre en español), esBisiesto (calcula si un año es bisiesto), estado_alquiler (evalúa si un alquiler fue devuelto con retraso, puntualidad o adelanto) y numero_alquileres (devuelve el número de alquileres en un mes y año concretos).
- **Triggers:** tr_alquiler_insert_actualiza_vehiculo (al crear un alquiler, el vehículo pasa automáticamente a 'alquilado') y tr_alquiler_update_estado (al registrar la devolución, el contrato pasa a 'Completado' y el vehículo vuelve a 'disponible').
- **Cursor:** listar_vehiculos_alquilados recorre todos los vehículos actualmente alquilados y muestra su matrícula, marca, modelo y el cliente que lo tiene.

Además, incluyo el diagrama entidad-relación.

CLASES PRINCIPALES CREADAS

Las clases principales del proyecto son los DTOs: Cliente, Vehiculo, Empleado, Alquiler y Categoria.

Herencia

En App.java, al crear un vehículo se ofrece la posibilidad de elegir entre diferentes tipos de combustible, incluyendo "Electrico". Aunque en el repositorio no hay una clase VehiculoElectrico, el diseño permite extender la clase Vehiculo en el para añadir atributos como la autonomía de la batería, tal como se indica en la memoria del proyecto. El campo combustible ya acepta el valor "electrico" como una opción más.

Interfaces.

Los DAOs implementan implícitamente los métodos CRUD: crear, leer, actualizar y eliminar.

Excepciones personalizadas.

El sistema incluye dos excepciones:

- **ClienteNoEncontradoException:** se lanza cuando se busca un cliente que no existe en la base de datos.
- **VehiculoNoDisponibleException:** se lanza cuando se intenta alquilar un vehículo que no está disponible.

Estas excepciones permiten manejar los errores de negocio de forma clara y controlada.

DEMOSTRACIÓN PRÁCTICA

La clase App contiene el método main que inicia el sistema, establece la conexión con la base de datos con ConexionBD.conectar() y muestra el menú principal con las opciones:

1. Gestión Clientes
2. Gestión Vehículos
3. Gestión Alquileres
4. Gestión Empleados
5. Historial de alquileres de un cliente

Dar de alta un cliente: Selecciono la opción 1 del menú. El sistema pide DNI, nombre, teléfono, email, dirección y número de carnet. Los datos se guardan en la tabla CLIENTE mediante ClienteD.crearCliente().

Listar vehículos disponibles: Accedo a la gestión de vehículos. El método VehiculoD.listarTodos() recupera todos los vehículos de la tabla VEHICULO y los muestra por pantalla.

Crear un alquiler: Introduzco el DNI del cliente, la matrícula del vehículo, la fecha de inicio y la fecha prevista de devolución. Si el vehículo está disponible, se crea el contrato y el trigger tr_alquiler_insert_actualiza_vehiculo cambia automáticamente su estado a "alquilado".

Excepción VehiculoNoDisponibleException: Si intento alquilar el mismo vehículo otra vez, el sistema lanza la excepción personalizada VehiculoNoDisponibleException con el mensaje correspondiente.

Registrar una devolución: Introduzco el ID del alquiler y la fecha real de devolución. El trigger `tr_alquiler_update_estado` actualiza el estado del contrato a "Completado" y cambia el estado del vehículo a "disponible". Además, la función `estado_alquiler` calcula si ha habido retraso, puntualidad o adelanto.

Excepción `ClienteNoEncontradoException`: Si busco un cliente con un DNI que no existe, el sistema lanza `ClienteNoEncontradoException` y muestra un mensaje de error.

Todo el código fuente está disponible en el repositorio de GitHub:
<https://github.com/MarinaDP2006/PROYECT-FINAL-RUEDAFACIL>

CIERRE

En resumen, RuedaFácil es una aplicación de consola desarrollada en Java con MySQL que demuestra:

- El uso del patrón DAO/DTO para separar la lógica de acceso a datos.
- La implementación de un CRUD completo para clientes, vehículos, empleados y alquileres.
- El manejo de colecciones, streams, herencia y excepciones personalizadas en Java.
- El uso de procedimientos almacenados, funciones, triggers y cursores en MySQL para automatizar procesos y garantizar la integridad de los datos.

Como propuestas de mejora, se podría añadir una interfaz gráfica (JavaFX o Swing) para hacer la aplicación más visual y sencilla, un sistema de pagos integrado, o una aplicación móvil para que los clientes puedan consultar su historial y hacer reservas.

CAPTURAS DE UNA APLICACIÓN DE CONSOLA

Qué capturar:

- Menú principal.
- Alta de cliente (introduciendo datos).
- Listado de vehículos disponibles.
- Creación de un alquiler (todo correcto).
- Mensaje de error: vehículo no disponible.
- Registro de devolución.
- Mensaje de error: cliente no encontrado.

3. MANUAL DE USUARIO PASO A PASO (con imágenes)

3.2.1. Menú principal

Al ejecutar App_Main.java, aparece la siguiente pantalla:

IMAGEN 1: Menú principal de RuedaFácil

El usuario debe escribir el número de la opción deseada y pulsar Enter. Las opciones son:

1. Gestión Clientes
2. Gestión Vehículos
3. Gestión Alquileres
4. Gestión Empleados
5. Historial de alquileres de un cliente
6. Salir

3.2.2. Gestión de clientes – Añadir cliente

Seleccionamos la opción 1 y luego 1 para añadir cliente. El sistema pide los datos uno a uno:

IMAGEN2: Introduciendo un nuevo cliente

Rellenamos:

- DNI
- Nombre completo
- Teléfono
- Correo electrónico
- Dirección
- Número de carnet de conducir

Al final, el sistema confirma el alta

3.2.3. Gestión de vehículos – Listar vehículos disponibles

Desde el menú principal elegimos 2 (Vehículos) y luego 4 (Consultar vehículos disponibles por categoría). El sistema muestra algo como:

IMAGEN 3: Vehículos disponibles en la categoría "turismo"

3.2.4. Gestión de alquileres – Crear un alquiler

Seleccionamos 3 (Alquiler) y luego 1 (Nuevo alquiler). El programa solicita:

- DNI del cliente
- Matrícula del vehículo
- Fecha de inicio (AAAA-MM-DD)
- Fecha prevista de devolución

IMAGEN 4: Creando un nuevo contrato de alquiler

Si el vehículo está disponible, se crea el alquiler y su estado pasa a "activo".

3.2.5. Excepción – Vehículo no disponible

Si intentamos alquilar un vehículo que ya está alquilado, el programa lanza una excepción personalizada:

IMAGEN 5: Mensaje de error al intentar alquilar un coche ya alquilado

3.2.6. Gestión de alquileres – Registrar devolución

Opción 3 → 3 (Registrar devolución). Pedirá el ID del alquiler y la fecha real de devolución. El sistema calcula si hay retraso y actualiza el estado del vehículo a "disponible".

IMAGEN 6: Registro de devolución con retraso de 2 días

3.2.7. Excepción – Cliente no encontrado

En cualquier pantalla donde se pida un DNI, si el cliente no existe, se muestra:

IMAGEN 7: Error al buscar un cliente que no está registrado

(Repite estructura similar para Gestión de empleados e Historial de alquileres)